

# Color Clash — System Architecture & Backend Schema

Rebuild v2.0 • Service topology, realtime design, PostgreSQL schema, Redis keyspace

Dhoola • Geeks for Games

18 September 2026

## Contents

---

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>1. Document control</b>                                          | <b>2</b>  |
| 1.1 Scope . . . . .                                                 | 2         |
| 1.2 The starting point . . . . .                                    | 2         |
| <b>2. Architecture principles</b>                                   | <b>2</b>  |
| <b>3. Service topology</b>                                          | <b>3</b>  |
| 3.1 Service responsibilities . . . . .                              | 3         |
| 3.2 Why match nodes are sticky . . . . .                            | 4         |
| <b>4. Realtime match design</b>                                     | <b>4</b>  |
| 4.1 Command path . . . . .                                          | 4         |
| 4.2 Room ownership and lifecycle . . . . .                          | 4         |
| 4.3 Failure behaviour . . . . .                                     | 4         |
| 4.4 RNG continuity . . . . .                                        | 5         |
| <b>5. Matchmaking</b>                                               | <b>5</b>  |
| 5.1 Queue model . . . . .                                           | 5         |
| 5.2 Rules . . . . .                                                 | 5         |
| <b>6. Data model overview</b>                                       | <b>5</b>  |
| <b>7. PostgreSQL schema</b>                                         | <b>6</b>  |
| 7.1 Enumerated types . . . . .                                      | 6         |
| 7.2 Identity . . . . .                                              | 7         |
| 7.3 Profile . . . . .                                               | 8         |
| 7.4 Economy — balances and ledger . . . . .                         | 8         |
| 7.5 Catalogue, inventory and purchases . . . . .                    | 9         |
| 7.6 Matches . . . . .                                               | 10        |
| 7.7 Stats, ratings and leaderboards . . . . .                       | 11        |
| 7.8 Progression: missions, streaks, seasons, achievements . . . . . | 12        |
| 7.9 Social and moderation . . . . .                                 | 14        |
| 7.10 Operational tables . . . . .                                   | 14        |
| <b>8. Redis keyspace</b>                                            | <b>15</b> |
| <b>9. REST API surface</b>                                          | <b>16</b> |
| <b>10. Match result write path</b>                                  | <b>16</b> |

|                                           |           |
|-------------------------------------------|-----------|
| <b>11. Scaling</b>                        | <b>17</b> |
| 11.1 Capacity assumptions . . . . .       | 17        |
| 11.2 Scaling levers . . . . .             | 17        |
| 11.3 Multi-region . . . . .               | 17        |
| <b>12. Security architecture</b>          | <b>17</b> |
| <b>13. Migration and evolution policy</b> | <b>18</b> |
| <b>14. Environments</b>                   | <b>18</b> |
| <b>15. Operational runbooks</b>           | <b>18</b> |

## 1. Document control

| Field                | Value                                |
|----------------------|--------------------------------------|
| Document             | System Architecture & Backend Schema |
| Product              | Color Clash                          |
| Release              | v2.0 — "Rebuild"                     |
| Status               | Draft for approval                   |
| Depends on           | PRD v2.0, TRD v2.0                   |
| Database             | PostgreSQL 16                        |
| Cache / coordination | Redis 7                              |

### 1.1 Scope

Service topology, the realtime match design, matchmaking, the complete persistent data model with DDL, the Redis keyspace, scaling and failure behaviour, and migration policy.

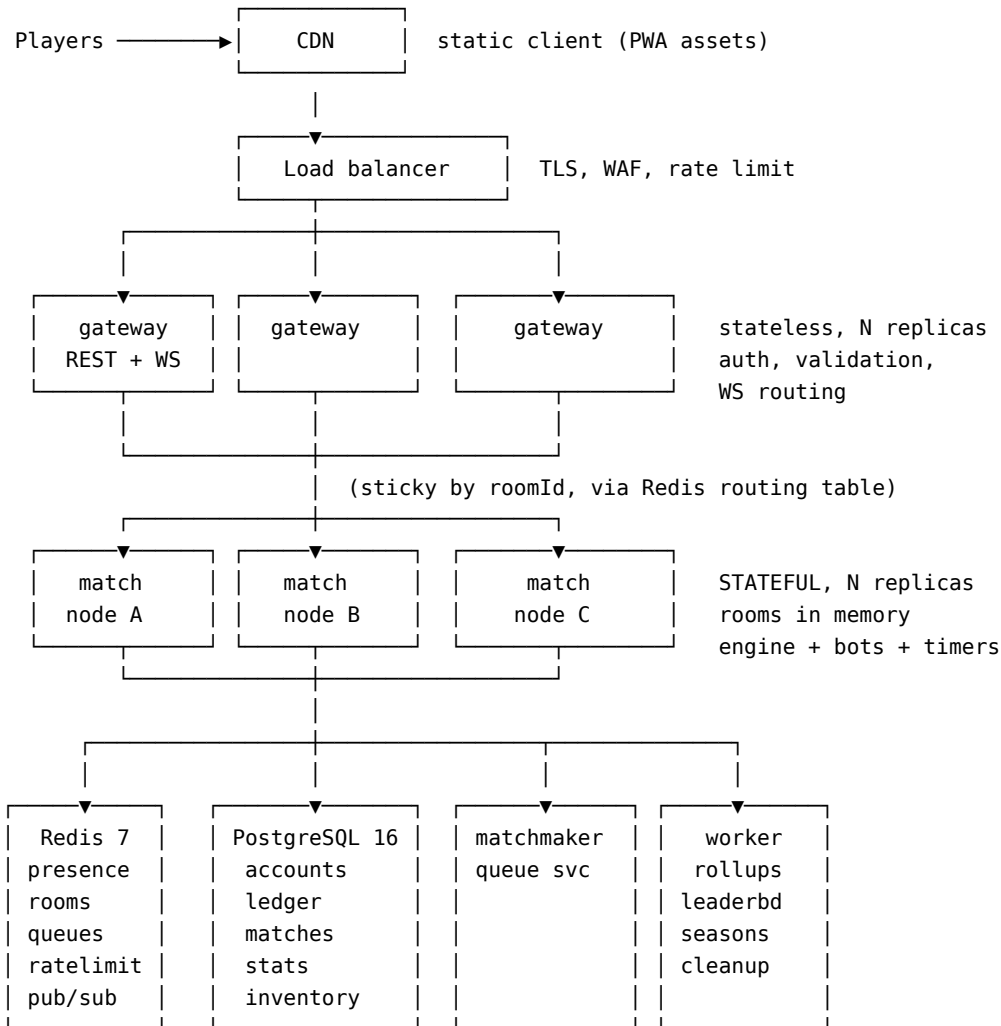
### 1.2 The starting point

v1.0 had no backend in any meaningful sense: one Node process, an in-memory Map of rooms, one JSON file per live room deleted 90 seconds after the last human left, and no database, no accounts and no authentication. Everything in this document is new, except the MatchRoom design — transport-agnostic, engine-driven, event-emitting — which was correct and is preserved.

## 2. Architecture principles

1. **One authority per fact.** Match truth lives in the match service. Account truth lives in PostgreSQL. Nothing is authoritative in two places.
2. **Stateful where it must be, stateless everywhere else.** Match nodes hold live rooms in memory and are therefore sticky. Every other service is horizontally scalable and holds nothing.
3. **Durability is separate from recovery.** Redis holds the live room so a process restart does not kill a match. PostgreSQL holds the permanent record so a finished match still exists next year.
4. **Money and progression are ledgered.** Every balance change is an append-only entry with a reason. A balance is a materialised view of its ledger and must be reconstructible from it.
5. **Blast radius is bounded.** Losing a match node loses at most the rooms on that node, and those rooms resume elsewhere from their checkpoint.
6. **Schema changes expand before they contract.** Every migration is compatible with the previously deployed version.

### 3. Service topology



#### 3.1 Service responsibilities

| Service    | Stateful?           | Responsibilities                                                                                                                                                                                                                                  |
|------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| gateway    | No                  | TLS termination behind the LB, REST API, authentication, token refresh, schema validation of every inbound message, WebSocket upgrade with Origin check, routing a socket to the match node that owns its room, fan-out of room events to sockets |
| match      | Yes                 | Owns live MatchRooms in memory. Runs reduce, the bot scheduler, the turn clock, invariant assertion. Checkpoints to Redis. Writes the completed match record to PostgreSQL.                                                                       |
| matchmaker | No (state in Redis) | Queue management per ruleset and mode, party grouping, rating bucketing, bot backfill at the SLA boundary, seat assignment, room creation request                                                                                                 |
| worker     | No                  | Scheduled jobs: stats rollups, leaderboard recomputation, season transitions, mission resets, expired-room cleanup, ledger reconciliation, data-deletion requests                                                                                 |

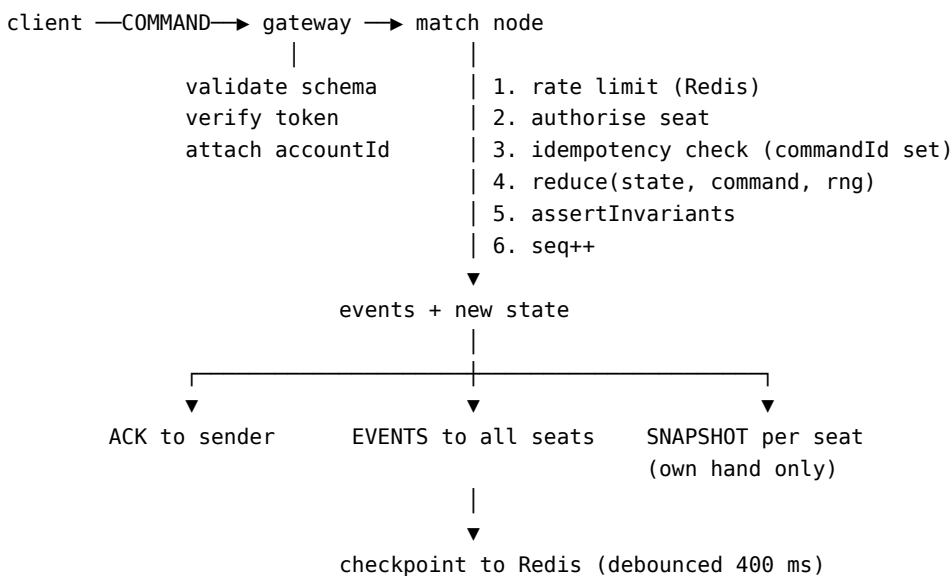
### 3.2 Why match nodes are sticky

A live match is a piece of hot mutable state with a 30-second decision clock. It is not a request. Routing every command through a shared store would add a round trip per command and make the invariant assertions racy. So a room is owned by exactly one node at a time, Redis holds the roomId → nodeId routing table, and gateways forward accordingly.

**Ownership is leased**, not permanent: each node renews a lease on its rooms every 5 seconds with a 15-second TTL. A node that dies stops renewing, the lease expires, and another node claims the room and restores it from its checkpoint.

## 4. Realtime match design

### 4.1 Command path



### 4.2 Room ownership and lifecycle

| Phase     | Behaviour                                                                                                                                                               |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Create    | Matchmaker or a player requests a room; the least-loaded eligible match node claims it and allocates a code via a Redis SET NX, so codes are unique <b>cluster-wide</b> |
| Lobby     | Seats, ready flags, host controls. Host-only actions are SET_VERSION, SET_CONFIG, START_GAME, ADD_BOT, REMOVE_SEAT. READY applies to the caller's own seat only.        |
| Playing   | Engine-driven. Turn clock armed per decision. Checkpoint every 40 commands.                                                                                             |
| Round end | Scores applied, standings broadcast, rematch offered                                                                                                                    |
| Match end | Match record written to PostgreSQL; rewards ledgered; stats enqueued                                                                                                    |
| Grace     | 90 s after the last connected human leaves                                                                                                                              |
| Closed    | Redis room key deleted, code released, routing entry removed                                                                                                            |

### 4.3 Failure behaviour

| Failure                | Behaviour                                                                                                                                                                                                                            |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| One socket drops       | Seat held, marked disconnected. Turn clock continues; timeouts auto-play.                                                                                                                                                            |
| All sockets drop       | Room held for 90 s, then closed.                                                                                                                                                                                                     |
| Match node crashes     | Lease expires within 15 s. Another node claims each room, restores from {checkpoint, commands[], rngCursor}, and broadcasts a SNAPSHOT with resumed: true. Clients reconnect through the normal backoff and land in the right place. |
| Redis unavailable      | New rooms refused with a clear message; <b>existing in-memory rooms continue to play</b> , checkpointing is queued. Matchmaking is suspended.                                                                                        |
| PostgreSQL unavailable | Matches continue and complete. Match records and reward ledger entries queue in a durable local outbox and are drained when the database returns. <b>A match is never blocked on a write to PostgreSQL.</b>                          |
| Deploy                 | Nodes drain: refuse new rooms, finish or checkpoint existing ones, send SERVER_DRAINING and close with 1001 so clients reconnect immediately rather than backing off.                                                                |

#### 4.4 RNG continuity

The RNG cursor is part of every checkpoint. This is the correction of a v1.0 defect in which restore re-ran the deal (consuming the RNG) and then jumped to a later checkpoint without advancing the cursor, so any post-checkpoint reshuffle diverged from the original match. A regression test pins it: checkpoint, restore, force draw-pile exhaustion, assert the reshuffle matches the uninterrupted run.

## 5. Matchmaking

### 5.1 Queue model

One queue per (ruleset, mode, region). Entries are Redis sorted sets scored by enqueue timestamp, with the rating bucket in the member key for ranked queues.

Player joins queue

- enqueue (accountId, ruleset, mode, rating, region, partyId?, enqueuedAt)
- matchmaker tick (every 500 ms per queue):
  - group by rating bucket, widening the bucket by  $\pm 50$  every 5 s
  - form a table of 4 preferred, 3 or 2 acceptable
  - at  $t = 30$  s: backfill remaining seats with bots and seat anyway
- reserve seats, request a room from the least-loaded match node
- push QUEUE\_MATCHED to each player's gateway via Redis pub/sub
- 10 s acceptance window; a non-accepting player is dropped and requeued once

### 5.2 Rules

- A party (P2 feature) is seated together or not at all.
- Rating bucketing applies only to ranked queues; casual queues match on wait time alone.
- **Bot backfill is a designed outcome, not a failure.** The client is told plainly.
- A player who is dropped for not accepting is requeued at their original timestamp, once.
- Queue depth, wait-time percentiles and backfill rate are first-class metrics.

## 6. Data model overview

accounts —1:N— auth\_identities

```

|
├──1:1── profiles
├──1:N── refresh_tokens
├──1:1── currency_balances —derived from— ledger_entries
├──1:N── inventory_items —N:1── catalog_items
├──1:N── purchases
├──1:N── match_players —N:1── matches —1:N── match_commands
├──1:N── player_stats
├──1:1── ratings —1:N── rating_history
├──1:N── mission_progress —N:1── missions
├──1:1── streaks
├──1:N── season_progress —N:1── seasons
├──1:N── account_achievements —N:1── achievements
├──1:N── friendships
├──1:N── reports (as reporter / as subject)
└──1:N── moderation_actions

```

---

## 7. PostgreSQL schema

Conventions used throughout:

- Primary keys are UUID v7 (time-ordered, so they index well) unless noted.
- Every table has `created_at` timestampz NOT NULL DEFAULT now().
- Mutable tables have `updated_at` timestampz NOT NULL DEFAULT now() maintained by a trigger.
- Money-like and currency values are bigint, never floating point.
- Enumerations are PostgreSQL enum types so an invalid value cannot be stored.
- Deletion is soft (`deleted_at`) wherever a foreign key would otherwise cascade into history.

### 7.1 Enumerated types

```

CREATE TYPE game_version AS ENUM
('CLASSIC', 'FLIP', 'MAYHEM', 'ALL_WILD', 'FLEX');

CREATE TYPE match_mode AS ENUM
('LOCAL', 'PASS_AND_PLAY', 'PRIVATE_ROOM', 'QUICK_MATCH', 'RANKED');

CREATE TYPE match_outcome AS ENUM
('COMPLETED', 'ABANDONED', 'ABORTED');

CREATE TYPE seat_result AS ENUM
('WON', 'LOST', 'ELIMINATED', 'DISCONNECTED', 'CONVERTED_TO_BOT');

CREATE TYPE bot_tier AS ENUM ('EASY', 'NORMAL', 'HARD', 'CHAOS');

CREATE TYPE currency AS ENUM ('COINS', 'GEMS');

CREATE TYPE ledger_reason AS ENUM (
'MATCH_REWARD', 'WIN_BONUS', 'DAILY_MISSION', 'LOGIN_STREAK',
'ACHIEVEMENT', 'SEASON_TIER', 'PURCHASE', 'IAP_GRANT',
'STORE_SPEND', 'AD_REWARD', 'REFUND', 'ADMIN_ADJUSTMENT',
'SIGNUP_GRANT', 'COMPENSATION'
);

CREATE TYPE item_kind AS ENUM
('CARD_BACK', 'TABLE_THEME', 'AVATAR', 'PROFILE_FRAME',
'EMOTE_PACK', 'SOUND_PACK', 'TITLE');

CREATE TYPE account_status AS ENUM
('ACTIVE', 'SUSPENDED', 'BANNED', 'DELETION_PENDING', 'DELETED');

```

```

CREATE TYPE identity_provider AS ENUM
('GUEST', 'PASSWORD', 'GOOGLE', 'APPLE');

CREATE TYPE moderation_kind AS ENUM
('WARNING', 'NAME_RESET', 'CHAT_MUTE', 'SUSPENSION', 'BAN');

```

## 7.2 Identity

```

CREATE TABLE accounts (
  id          uuid PRIMARY KEY,
  status      account_status NOT NULL DEFAULT 'ACTIVE',
  is_guest    boolean        NOT NULL DEFAULT true,
  -- Declared at signup. Stored as a year only: enough for an age gate,
  -- not enough to be a date of birth on file.
  birth_year  smallint,
  country_code char(2),
  locale      text          NOT NULL DEFAULT 'en',
  consent_analytics boolean  NOT NULL DEFAULT false,
  consent_ads  boolean      NOT NULL DEFAULT false,
  consent_recorded_at timestampz,
  last_seen_at timestampz,
  created_at   timestampz  NOT NULL DEFAULT now(),
  updated_at   timestampz  NOT NULL DEFAULT now(),
  deleted_at   timestampz,
  CONSTRAINT birth_year_sane CHECK (birth_year IS NULL
    OR birth_year BETWEEN 1900 AND EXTRACT(YEAR FROM now())::int)
);

CREATE INDEX accounts_last_seen_idx ON accounts (last_seen_at DESC)
  WHERE deleted_at IS NULL;

-- One row per sign-in method. A guest account has exactly one GUEST row;
-- upgrading adds a PASSWORD or GOOGLE row and flips accounts.is_guest.
CREATE TABLE auth_identities (
  id          uuid PRIMARY KEY,
  account_id  uuid NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  provider    identity_provider NOT NULL,
  -- Provider subject: the device install id for GUEST, the lowercased
  -- email for PASSWORD, the OIDC `sub` for GOOGLE/APPLE.
  subject     text NOT NULL,
  email       citext,
  email_verified boolean NOT NULL DEFAULT false,
  -- Argon2id encoded hash. NULL for every provider except PASSWORD.
  password_hash text,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  UNIQUE (provider, subject)
);

CREATE UNIQUE INDEX auth_identities_email_idx
  ON auth_identities (email) WHERE email IS NOT NULL;
CREATE INDEX auth_identities_account_idx ON auth_identities (account_id);

CREATE TABLE refresh_tokens (
  id          uuid PRIMARY KEY,
  account_id  uuid NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  -- SHA-256 of the token. The token itself is never stored.
  token_hash  bytea NOT NULL UNIQUE,
  -- Rotation family. Replaying a rotated token revokes the whole family.
  family_id   uuid NOT NULL,
  parent_id   uuid REFERENCES refresh_tokens(id),
  device_label text,
  user_agent  text,
  issued_at   timestampz NOT NULL DEFAULT now(),
  expires_at  timestampz NOT NULL,
  revoked_at  timestampz,
  revoked_reason text
);

```

```
CREATE INDEX refresh_tokens_account_idx ON refresh_tokens (account_id)
WHERE revoked_at IS NULL;
CREATE INDEX refresh_tokens_family_idx ON refresh_tokens (family_id);
CREATE INDEX refresh_tokens_expiry_idx ON refresh_tokens (expires_at);
```

### 7.3 Profile

```
CREATE TABLE profiles (
  account_id      uuid PRIMARY KEY REFERENCES accounts(id) ON DELETE CASCADE,
  -- 1-16 characters. Not globally unique; (display_name, discriminator) is.
  display_name    text NOT NULL,
  discriminator    char(4) NOT NULL,
  -- Case- and confusable-folded form, for the uniqueness constraint and
  -- for blacklist matching.
  name_normalised text NOT NULL,
  avatar_item_id  uuid REFERENCES catalog_items(id),
  frame_item_id   uuid REFERENCES catalog_items(id),
  card_back_id    uuid REFERENCES catalog_items(id),
  table_theme_id  uuid REFERENCES catalog_items(id),
  title_item_id   uuid REFERENCES catalog_items(id),
  level           integer NOT NULL DEFAULT 1,
  xp              bigint NOT NULL DEFAULT 0,
  name_changed_at timestampz,
  created_at      timestampz NOT NULL DEFAULT now(),
  updated_at      timestampz NOT NULL DEFAULT now(),
  CONSTRAINT display_name_length CHECK (char_length(display_name) BETWEEN 1 AND 16),
  CONSTRAINT level_positive CHECK (level >= 1),
  CONSTRAINT xp_non_negative CHECK (xp >= 0)
);

CREATE UNIQUE INDEX profiles_handle_idx
ON profiles (name_normalised, discriminator);
CREATE INDEX profiles_name_search_idx
ON profiles USING gin (name_normalised gin_trgm_ops);

CREATE TABLE name_blocklist (
  pattern      text PRIMARY KEY,
  is_regex     boolean NOT NULL DEFAULT false,
  severity     smallint NOT NULL DEFAULT 1,
  created_at   timestampz NOT NULL DEFAULT now()
);
```

### 7.4 Economy — balances and ledger

The ledger is the source of truth. `currency_balances` is a materialised running total maintained in the same transaction as the ledger insert, so a read is one row rather than a sum over history — but it must always equal that sum.

```
CREATE TABLE currency_balances (
  account_id      uuid NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  currency        currency NOT NULL,
  balance         bigint NOT NULL DEFAULT 0,
  -- Ledger sequence this balance reflects. Detects a missed entry.
  last_entry_seq  bigint NOT NULL DEFAULT 0,
  updated_at      timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (account_id, currency),
  CONSTRAINT balance_non_negative CHECK (balance >= 0)
);

CREATE TABLE ledger_entries (
  id              uuid PRIMARY KEY,
  -- Global monotonic sequence, for reconciliation and replication.
  seq            bigint NOT NULL UNIQUE,
  account_id      uuid NOT NULL REFERENCES accounts(id),
  currency        currency NOT NULL,
  -- Signed. Positive credits, negative debits.
  amount         bigint NOT NULL,
```

```

balance_after bigint NOT NULL,
reason ledger_reason NOT NULL,
-- What caused it: a match id, a store sku, a mission id, a purchase id.
ref_type text,
ref_id uuid,
-- Caller-supplied key that makes a credit safe to retry.
idempotency_key text,
metadata jsonb NOT NULL DEFAULT '{}::jsonb,
created_at timestamptz NOT NULL DEFAULT now(),
CONSTRAINT amount_non_zero CHECK (amount <> 0),
CONSTRAINT balance_after_ok CHECK (balance_after >= 0)
);

CREATE UNIQUE INDEX ledger_idempotency_idx
ON ledger_entries (account_id, idempotency_key)
WHERE idempotency_key IS NOT NULL;
CREATE INDEX ledger_account_time_idx
ON ledger_entries (account_id, created_at DESC);
CREATE INDEX ledger_ref_idx ON ledger_entries (ref_type, ref_id);

```

The ledger is append-only. A BEFORE UPDATE OR DELETE trigger raises an exception on ledger\_entries. A mistake is corrected by a compensating entry with reason ADMIN\_ADJUSTMENT or REFUND, never by editing history.

#### Reconciliation job (nightly, worker):

```

SELECT b.account_id, b.currency, b.balance, COALESCE(SUM(l.amount), 0) AS ledger_sum
FROM currency_balances b
LEFT JOIN ledger_entries l
ON l.account_id = b.account_id AND l.currency = b.currency
GROUP BY b.account_id, b.currency, b.balance
HAVING b.balance <> COALESCE(SUM(l.amount), 0);

```

Any row returned is a Sev-2 incident.

## 7.5 Catalogue, inventory and purchases

```

CREATE TABLE catalog_items (
  id uuid PRIMARY KEY,
  sku text UNIQUE NOT NULL,
  kind item_kind NOT NULL,
  name text NOT NULL,
  description text,
  rarity smallint NOT NULL DEFAULT 1,
  -- NULL means not purchasable with that currency.
  price_coins bigint,
  price_gems bigint,
  -- Renderer parameters: palette, motif, texture. No binary assets in the DB.
  render_spec jsonb NOT NULL DEFAULT '{}::jsonb,
  season_id uuid REFERENCES seasons(id),
  available_from timestamptz,
  available_to timestamptz,
  is_active boolean NOT NULL DEFAULT true,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT price_sane CHECK (
    (price_coins IS NULL OR price_coins > 0) AND
    (price_gems IS NULL OR price_gems > 0)
  );

CREATE INDEX catalog_active_idx ON catalog_items (kind, is_active)
WHERE is_active;

CREATE TABLE inventory_items (
  account_id uuid NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  item_id uuid NOT NULL REFERENCES catalog_items(id),
  -- How it was obtained, for support and for audit.
  source ledger_reason NOT NULL,
  source_ref uuid,

```

```

    acquired_at timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (account_id, item_id)
);

CREATE INDEX inventory_account_idx ON inventory_items (account_id);

CREATE TABLE purchases (
    id          uuid PRIMARY KEY,
    account_id  uuid NOT NULL REFERENCES accounts(id),
    platform    text NOT NULL,           -- 'google_play' | 'web'
    -- Google Play order id, or the payment processor's reference.
    platform_order_id text NOT NULL,
    product_sku text NOT NULL,
    gems_granted bigint NOT NULL DEFAULT 0,
    price_minor  bigint NOT NULL,       -- smallest currency unit
    price_currency char(3) NOT NULL,
    -- Raw receipt kept for dispute handling; never returned to a client.
    receipt      jsonb NOT NULL,
    verified_at  timestamptz,
    refunded_at  timestamptz,
    created_at   timestamptz NOT NULL DEFAULT now(),
    UNIQUE (platform, platform_order_id)
);

CREATE INDEX purchases_account_idx ON purchases (account_id, created_at DESC);

```

**Purchase flow:** the client never grants anything. The store receipt is verified server-side against the platform, then a purchases row and a ledger\_entries row with reason IAP\_GRANT are written in one transaction, keyed by platform\_order\_id so a retry cannot double-grant.

## 7.6 Matches

```

CREATE TABLE matches (
    id          uuid PRIMARY KEY,
    room_code   char(5),
    version     game_version NOT NULL,
    mode        match_mode  NOT NULL,
    -- Reproduces the entire match together with match_commands.
    seed        text        NOT NULL,
    config      jsonb       NOT NULL,
    seat_count  smallint    NOT NULL,
    bot_count   smallint    NOT NULL DEFAULT 0,
    outcome     match_outcome NOT NULL,
    winner_account_id uuid  REFERENCES accounts(id),
    round_count smallint    NOT NULL DEFAULT 1,
    -- Final accepted command count; equals GameState.seq at the end.
    command_count integer   NOT NULL DEFAULT 0,
    duration_ms integer,
    server_version text     NOT NULL,
    protocol_version smallint NOT NULL,
    started_at  timestamptz NOT NULL,
    ended_at    timestamptz,
    created_at  timestamptz NOT NULL DEFAULT now(),
    CONSTRAINT seat_count_range CHECK (seat_count BETWEEN 2 AND 6),
    CONSTRAINT bot_count_range  CHECK (bot_count BETWEEN 0 AND seat_count)
) PARTITION BY RANGE (created_at);

-- Monthly partitions, created ahead by the worker.
CREATE TABLE matches_2026_09 PARTITION OF matches
    FOR VALUES FROM ('2026-09-01') TO ('2026-10-01');

CREATE INDEX matches_mode_time_idx ON matches (mode, created_at DESC);
CREATE INDEX matches_winner_idx   ON matches (winner_account_id, created_at DESC);
CREATE INDEX matches_room_code_idx ON matches (room_code) WHERE room_code IS NOT NULL;

CREATE TABLE match_players (
    match_id      uuid NOT NULL,
    seat_index    smallint NOT NULL,

```

```

-- NULL for a bot seat.
account_id      uuid REFERENCES accounts(id),
display_name    text NOT NULL,
is_bot          boolean NOT NULL DEFAULT false,
bot_tier        bot_tier,
is_host         boolean NOT NULL DEFAULT false,
result          seat_result NOT NULL,
final_position  smallint,
score           integer NOT NULL DEFAULT 0,
-- Value of cards still held at round end. Always >= 0.
hand_value      integer NOT NULL DEFAULT 0,
cards_left      smallint NOT NULL DEFAULT 0,
cards_played    smallint NOT NULL DEFAULT 0,
clash_calls     smallint NOT NULL DEFAULT 0,
clash_missed    smallint NOT NULL DEFAULT 0,
penalties_drawn smallint NOT NULL DEFAULT 0,
timeouts        smallint NOT NULL DEFAULT 0,
disconnects     smallint NOT NULL DEFAULT 0,
rating_before   integer,
rating_after    integer,
PRIMARY KEY (match_id, seat_index),
CONSTRAINT bot_has_no_account CHECK (NOT is_bot OR account_id IS NULL),
CONSTRAINT hand_value_non_negative CHECK (hand_value >= 0)
);

CREATE INDEX match_players_account_idx
ON match_players (account_id, match_id) WHERE account_id IS NOT NULL;

-- The command log. Seed + config + this = the whole match, replayable
-- through the deterministic reducer. Stored as one row per match to keep
-- the row count sane; the array is append-only during play.
CREATE TABLE match_commands (
  match_id      uuid PRIMARY KEY,
  -- RNG cursor at the start, so replay is exact.
  rng_cursor     bigint NOT NULL,
  commands       jsonb NOT NULL, -- Command[]
  compressed     boolean NOT NULL DEFAULT false,
  byte_size      integer NOT NULL,
  created_at     timestamptz NOT NULL DEFAULT now()
);

```

**Retention:** match\_commands is retained 90 days (long enough for dispute analysis and replay); matches and match\_players are retained indefinitely because stats and history derive from them. A partition older than 90 days has its match\_commands rows dropped by the worker.

## 7.7 Stats, ratings and leaderboards

```

CREATE TABLE player_stats (
  account_id      uuid NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  -- NULL row = the all-rulesets aggregate.
  version         game_version,
  matches_played  integer NOT NULL DEFAULT 0,
  matches_won     integer NOT NULL DEFAULT 0,
  matches_abandoned integer NOT NULL DEFAULT 0,
  rounds_played   integer NOT NULL DEFAULT 0,
  cards_played    bigint NOT NULL DEFAULT 0,
  clash_calls     integer NOT NULL DEFAULT 0,
  clash_missed    integer NOT NULL DEFAULT 0,
  eliminations    integer NOT NULL DEFAULT 0,
  total_score     bigint NOT NULL DEFAULT 0,
  current_streak  integer NOT NULL DEFAULT 0,
  longest_streak  integer NOT NULL DEFAULT 0,
  fastest_win_ms  integer,
  last_played_at  timestamptz,
  updated_at      timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (account_id, COALESCE(version, 'CLASSIC'::game_version))
);

```

Note: PostgreSQL will not accept an expression in a primary key. In the implementation this becomes a NOT NULL version column with a sentinel 'ALL' member added to the enum, plus PRIMARY KEY (account\_id, version). It is written above in the conceptual form for readability.

```
CREATE TABLE ratings (
  account_id  uuid NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  version     game_version NOT NULL,
  -- Glicko-2
  rating      integer NOT NULL DEFAULT 1500,
  deviation   integer NOT NULL DEFAULT 350,
  volatility   numeric(6,5) NOT NULL DEFAULT 0.06000,
  tier         text NOT NULL DEFAULT 'BRONZE',
  games Rated integer NOT NULL DEFAULT 0,
  season_id   uuid REFERENCES seasons(id),
  updated_at  timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (account_id, version)
);

CREATE INDEX ratings_leaderboard_idx ON ratings (version, rating DESC)
WHERE games Rated >= 10;

CREATE TABLE rating_history (
  id          uuid PRIMARY KEY,
  account_id  uuid NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  match_id    uuid NOT NULL,
  version     game_version NOT NULL,
  rating_before integer NOT NULL,
  rating_after integer NOT NULL,
  deviation_after integer NOT NULL,
  created_at  timestampz NOT NULL DEFAULT now()
);

CREATE INDEX rating_history_account_idx
ON rating_history (account_id, created_at DESC);

-- Recomputed on a schedule, never on read.
CREATE TABLE leaderboard_entries (
  board_key   text NOT NULL, -- e.g. 'weekly:CLASSIC:global:2026-W38'
  rank        integer NOT NULL,
  account_id  uuid NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  display_name text NOT NULL,
  discriminator char(4) NOT NULL,
  score       bigint NOT NULL,
  metadata    jsonb NOT NULL DEFAULT '{}'::jsonb,
  computed_at timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (board_key, rank)
);

CREATE INDEX leaderboard_account_idx ON leaderboard_entries (account_id, board_key);
```

## 7.8 Progression: missions, streaks, seasons, achievements

```
CREATE TABLE missions (
  id          uuid PRIMARY KEY,
  code        text UNIQUE NOT NULL,
  title       text NOT NULL,
  description  text NOT NULL,
  cadence     text NOT NULL, -- 'DAILY' | 'WEEKLY' | 'EVENT'
  -- Declarative goal, evaluated server-side against match results.
  -- e.g. {"metric":"matches_won","version":"MAYHEM","target":3}
  criteria    jsonb NOT NULL,
  reward_currency currency,
  reward_amount bigint,
  reward_item_id uuid REFERENCES catalog_items(id),
  reward_xp   integer NOT NULL DEFAULT 0,
  is_active   boolean NOT NULL DEFAULT true,
  created_at  timestampz NOT NULL DEFAULT now()
);
```

```

CREATE TABLE mission_progress (
  account_id  uuid NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  mission_id  uuid NOT NULL REFERENCES missions(id),
  -- The reset bucket this progress belongs to (UTC date or ISO week).
  period_key  text NOT NULL,
  progress    integer NOT NULL DEFAULT 0,
  target      integer NOT NULL,
  completed_at timestamptz,
  claimed_at  timestamptz,
  updated_at  timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (account_id, mission_id, period_key)
);

CREATE INDEX mission_progress_open_idx
  ON mission_progress (account_id, period_key) WHERE claimed_at IS NULL;

CREATE TABLE streaks (
  account_id  uuid PRIMARY KEY REFERENCES accounts(id) ON DELETE CASCADE,
  current_days integer NOT NULL DEFAULT 0,
  longest_days integer NOT NULL DEFAULT 0,
  last_claim_date date,
  updated_at  timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE seasons (
  id          uuid PRIMARY KEY,
  code        text UNIQUE NOT NULL,
  name        text NOT NULL,
  starts_at   timestamptz NOT NULL,
  ends_at     timestamptz NOT NULL,
  -- [{tier, xp_required, free_item_id, paid_item_id, coins}]
  tiers       jsonb NOT NULL,
  is_active   boolean NOT NULL DEFAULT false,
  CONSTRAINT season_window CHECK (ends_at > starts_at)
);

CREATE TABLE season_progress (
  account_id  uuid NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  season_id   uuid NOT NULL REFERENCES seasons(id),
  xp          bigint NOT NULL DEFAULT 0,
  tier         integer NOT NULL DEFAULT 0,
  has_paid_track boolean NOT NULL DEFAULT false,
  claimed_tiers integer[] NOT NULL DEFAULT '{}',
  updated_at  timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (account_id, season_id)
);

CREATE TABLE achievements (
  id          uuid PRIMARY KEY,
  code        text UNIQUE NOT NULL,
  title       text NOT NULL,
  description text NOT NULL,
  criteria    jsonb NOT NULL,
  reward_item_id uuid REFERENCES catalog_items(id),
  reward_xp    integer NOT NULL DEFAULT 0,
  is_secret    boolean NOT NULL DEFAULT false
);

CREATE TABLE account_achievements (
  account_id  uuid NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  achievement_id uuid NOT NULL REFERENCES achievements(id),
  progress    integer NOT NULL DEFAULT 0,
  unlocked_at timestamptz,
  PRIMARY KEY (account_id, achievement_id)
);

```

## 7.9 Social and moderation

```
CREATE TABLE friendships (
  -- Stored once per pair with account_a < account_b, so a pair cannot
  -- exist twice with the roles swapped.
  account_a    uuid NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  account_b    uuid NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  requested_by  uuid NOT NULL REFERENCES accounts(id),
  status        text NOT NULL DEFAULT 'PENDING', -- PENDING|ACCEPTED|BLOCKED
  created_at    timestampz NOT NULL DEFAULT now(),
  updated_at    timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (account_a, account_b),
  CONSTRAINT ordered_pair CHECK (account_a < account_b)
);

CREATE INDEX friendships_b_idx ON friendships (account_b, status);

CREATE TABLE reports (
  id            uuid PRIMARY KEY,
  reporter_id   uuid NOT NULL REFERENCES accounts(id),
  subject_id    uuid NOT NULL REFERENCES accounts(id),
  match_id      uuid,
  category       text NOT NULL, -- NAME | CHAT | HARASSMENT | CHEATING | OTHER
  detail        text,
  evidence       jsonb NOT NULL DEFAULT '{}'::jsonb,
  status        text NOT NULL DEFAULT 'OPEN',
  resolved_at    timestampz,
  resolved_by   text,
  created_at    timestampz NOT NULL DEFAULT now(),
  CONSTRAINT no_self_report CHECK (reporter_id <> subject_id)
);

CREATE INDEX reports_subject_idx ON reports (subject_id, created_at DESC);
CREATE INDEX reports_open_idx ON reports (status, created_at)
  WHERE status = 'OPEN';

CREATE TABLE moderation_actions (
  id            uuid PRIMARY KEY,
  account_id    uuid NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  kind          moderation_kind NOT NULL,
  reason        text NOT NULL,
  report_id     uuid REFERENCES reports(id),
  actor         text NOT NULL, -- moderator id or 'SYSTEM'
  expires_at    timestampz, -- NULL = permanent
  created_at    timestampz NOT NULL DEFAULT now()
);

CREATE INDEX moderation_active_idx ON moderation_actions (account_id)
  WHERE expires_at IS NULL OR expires_at > now();
```

## 7.10 Operational tables

```
-- Durable outbox, so a match never blocks on PostgreSQL being reachable.
CREATE TABLE outbox_events (
  id            uuid PRIMARY KEY,
  topic         text NOT NULL,
  payload       jsonb NOT NULL,
  attempts      smallint NOT NULL DEFAULT 0,
  available_at   timestampz NOT NULL DEFAULT now(),
  processed_at   timestampz,
  last_error    text,
  created_at    timestampz NOT NULL DEFAULT now()
);

CREATE INDEX outbox_pending_idx ON outbox_events (available_at)
  WHERE processed_at IS NULL;

-- Self-service deletion, satisfied within 30 days.
CREATE TABLE deletion_requests (
```

```

account_id    uuid PRIMARY KEY REFERENCES accounts(id) ON DELETE CASCADE,
requested_at  timestampz NOT NULL DEFAULT now(),
scheduled_for timestampz NOT NULL,
completed_at  timestampz,
cancelled_at  timestampz
);

CREATE TABLE schema_migrations (
  version     text PRIMARY KEY,
  applied_at  timestampz NOT NULL DEFAULT now(),
  checksum    text NOT NULL
);

```

## 8. Redis keyspace

Redis holds everything that is hot, ephemeral, or needs to be shared across nodes. **Nothing in Redis is a source of truth for anything that must survive.**

| Key pattern                        | Type             | TTL    | Purpose                                                                              |
|------------------------------------|------------------|--------|--------------------------------------------------------------------------------------|
| room:{roomId}                      | Hash             | 6 h    | Live room checkpoint: config, seats, seed, rngCursor, checkpoint state, command tail |
| room:code:{CODE}                   | String           | 6 h    | Code → roomId. Allocated with SET NX, which is what makes codes unique cluster-wide  |
| room:owner:{roomId}                | String           | 15 s   | Routing table: roomId → nodeId. Renewed every 5 s by the owning node                 |
| room:seats:{roomId}                | Set              | 6 h    | accountIds seated, for reconnect routing                                             |
| node:{nodeId}:rooms                | Set              | 30 s   | Rooms a node claims, for load balancing and orphan detection                         |
| node:load                          | Sorted set       | —      | nodeId scored by room count, for least-loaded placement                              |
| presence:{accountId}               | String           | 60 s   | Current connection: gatewayId, roomId, since                                         |
| mm:queue:{version}:{mode}:{region} | Sorted set       | —      | Queue, scored by enqueue timestamp                                                   |
| mm:party:{partyId}                 | Set              | 30 m   | Party membership                                                                     |
| mm:pending:{accountId}             | String           | 10 s   | Acceptance window for a formed match                                                 |
| rl:{scope}:{id}                    | String (counter) | window | Rate limiting — sliding window per account, per IP, per connection                   |
| session:{accountId}                | Set              | 30 d   | Active refresh-token families, for global sign-out                                   |
| lock:{resource}                    | String           | 30 s   | Advisory locks (season rollover, leaderboard compute)                                |
| cache:profile:{accountId}          | String (JSON)    | 5 m    | Read-through profile cache                                                           |
| cache:lb:{boardKey}                | String (JSON)    | 5 m    | Leaderboard page cache                                                               |

Pub/sub channels:

| Channel                    | Publisher  | Subscriber  | Purpose                                          |
|----------------------------|------------|-------------|--------------------------------------------------|
| room:{roomId}:events       | match node | gateways    | Fan-out of room events to sockets                |
| account:{accountId}:notify | any        | gateways    | Seat takeover, moderation notice, friend request |
| mm:matched                 | matchmaker | gateways    | Queue result delivery                            |
| ops:drain                  | deploy     | match nodes | Begin draining                                   |

## 9. REST API surface

Small by design; the realtime surface is the WebSocket protocol.

| Method | Path                                    | Purpose                                              |
|--------|-----------------------------------------|------------------------------------------------------|
| POST   | /v1/auth/guest                          | Create a guest account, return a token pair          |
| POST   | /v1/auth/register                       | Email + password registration, or guest upgrade      |
| POST   | /v1/auth/login                          | Password login                                       |
| POST   | /v1/auth/oauth/:provider                | Google (later Apple) exchange                        |
| POST   | /v1/auth/refresh                        | Rotate the refresh token                             |
| POST   | /v1/auth/logout                         | Revoke this token family                             |
| POST   | /v1/auth/ws-ticket                      | Short-lived single-use ticket for the WS upgrade     |
| GET    | /v1/me                                  | Profile, balances, equipped cosmetics, consent state |
| PATCH  | /v1/me                                  | Display name, avatar, equipped cosmetics, settings   |
| DELETE | /v1/me                                  | Request account deletion                             |
| GET    | /v1/me/stats                            | Per-ruleset stats                                    |
| GET    | /v1/me/matches                          | Match history, paginated                             |
| GET    | /v1/matches/:id                         | One match with standings                             |
| GET    | /v1/matches/:id/replay                  | Seed, config and command log, if retained            |
| GET    | /v1/me/inventory                        | Owned items                                          |
| GET    | /v1/store                               | Active catalogue                                     |
| POST   | /v1/store/purchase                      | Spend coins or gems on an item                       |
| POST   | /v1/store/iap/verify                    | Verify a platform receipt and grant gems             |
| GET    | /v1/me/missions                         | Missions with progress                               |
| POST   | /v1/me/missions/:id/claim               | Claim a completed mission                            |
| GET    | /v1/leaderboards/:board                 | A leaderboard page                                   |
| GET    | /v1/friends · POST /v1/friends · DELETE | Friends (P2)                                         |
|        | /v1/friends/:id                         |                                                      |
| POST   | /v1/reports                             | Report a player                                      |
| GET    | /healthz · /readyz · /metrics           | Operations                                           |

Conventions: cursor pagination (?cursor=&limit=), ETag on cacheable resources, RFC 7807 problem responses, Idempotency-Key required on every mutating economy endpoint.

## 10. Match result write path

At MATCH\_ENDED the match node performs exactly one transaction, plus an outbox enqueue:

```
BEGIN
  INSERT matches                                -- one row
  INSERT match_players                          -- one row per seat
  INSERT match_commands                        -- seed + rng cursor + log
  INSERT ledger_entries (MATCH_REWARD, WIN_BONUS) -- per human seat
  UPDATE currency_balances                    -- same transaction
  INSERT outbox_events ('match.completed')      -- for the worker
COMMIT
```

Everything downstream — stats rollups, ratings, missions, season XP, achievements, leaderboards — is driven **asynchronously from the outbox**, not inline. This keeps the transaction small and bounded, and it means a slow rollup can never delay a player's result screen.

**Idempotency:** the outbox consumer is keyed by `match_id`, so a redelivery recomputes the same rollup rather than double-counting. Reward ledger entries use `idempotency_key = 'match:' || match_id || ':' || account_id`.

**Reward caps:** daily coin earnings are capped per account. The cap is evaluated in the same transaction by summing today's `MATCH_REWARD` and `WIN_BONUS` entries, so farming is pointless and the cap cannot be raced.

## 11. Scaling

### 11.1 Capacity assumptions

| Quantity                             | Assumption                                                        |
|--------------------------------------|-------------------------------------------------------------------|
| Average match                        | 4 seats, 6 minutes, ~120 accepted commands                        |
| Commands per second per active match | ~0.35                                                             |
| Snapshot fan-out per command         | seats × 1                                                         |
| Memory per live room                 | ~200 KB including the card table and command tail                 |
| Rooms per match node                 | 500 ( <code>CC_MAX_ROOMS_PER_NODE</code> ), ~100 MB of room state |
| Concurrent players per match node    | ~2,000                                                            |

### 11.2 Scaling levers

| Pressure                         | Lever                                                                                    |
|----------------------------------|------------------------------------------------------------------------------------------|
| More concurrent matches          | Add match nodes. Placement is least-loaded via <code>node:load</code> .                  |
| More connections, same matches   | Add gateways. They are stateless.                                                        |
| Read-heavy profile/stat traffic  | PostgreSQL read replicas plus the Redis read-through caches                              |
| Leaderboard cost                 | Precomputed on a schedule into <code>leaderboard_entries</code> , never computed on read |
| Match table growth               | Monthly partitions; <code>match_commands</code> dropped after 90 days                    |
| Queue latency at low concurrency | Merge regional queues below a concurrency floor; bot backfill at 30 s                    |

### 11.3 Multi-region

Single region at launch. The design that keeps a second region cheap: match nodes and their Redis are region-local, matchmaking queues are region-scoped, and PostgreSQL is a single primary with regional read replicas. Cross-region play is explicitly not supported — latency would make the 30-second turn clock unfair before it made anything else break.

## 12. Security architecture

| Layer      | Control                                                                                                                                                       |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Edge       | TLS, HSTS, WAF, per-IP connection and request limits                                                                                                          |
| Gateway    | Token verification on every request and on WS upgrade; Origin allowlist; Zod validation of every message; per-account and per-connection rate limits in Redis |
| Match node | Seat authorisation per command; host authorisation per lobby action; idempotency set; invariant assertion; frame size and buffer caps                         |
| Data       | Private hands never leave the node except in the owning seat's own snapshot; no token, password, email or hand in any log                                     |
| Storage    | Encryption at rest; passwords Argon2id; refresh tokens stored as SHA-256 hashes; receipts never returned to a client                                          |

| Layer        | Control                                                                                                                              |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Economy      | Server-side only; append-only ledger with an update/delete trigger; nightly reconciliation; daily earn caps                          |
| Access       | Least privilege database roles — the match service can write matches and ledger entries but cannot read <code>auth_identities</code> |
| Supply chain | Dependency and container scanning on every build; a known critical advisory fails the build                                          |

## 13. Migration and evolution policy

1. **Forward-only, versioned migrations** checked into the repo, applied before the new version starts.
2. **Expand → migrate → contract.** Add the new column nullable; deploy code that writes both; backfill; deploy code that reads the new; drop the old in a later release. No migration is ever compatible only with the version being deployed.
3. **No destructive migration in the same release as the code that stops using the column.**
4. CI applies every migration to a fresh database **and** to a snapshot of the previous release's schema.
5. A migration that cannot be rolled forward safely is not merged.
6. `match_commands` and `ledger_entries` shapes are versioned in their payloads, because they are replayed by code that may be newer than the data.

## 14. Environments

| Environment | Purpose                                                                             | Data            |
|-------------|-------------------------------------------------------------------------------------|-----------------|
| dev         | Local. Docker Compose: Postgres, Redis, all four services.                          | Seeded fixtures |
| staging     | Pre-production, identical topology at smaller scale. Load and chaos tests run here. | Synthetic only  |
| production  | Live                                                                                | Real            |

The same container images are promoted across environments; only configuration differs. Production data is never copied into a lower environment.

## 15. Operational runbooks

| Scenario                | Response                                                                                                                                                         |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Match node unresponsive | Lease expires in $\leq 15$ s; rooms are claimed and restored elsewhere. Confirm room count recovers on node: load; investigate the node.                         |
| Redis primary failover  | Existing rooms keep playing from memory. New rooms and matchmaking are refused with a clear message. Checkpoints queue and drain on recovery.                    |
| PostgreSQL unavailable  | Matches continue. Match records and ledger entries accumulate in the outbox. Verify the outbox drains fully before declaring recovery.                           |
| desync_detected fires   | <b>Sev-1.</b> Capture the diagnostic bundle (seed, config, command log, final state) by match id, replay locally, and treat any divergence as a rule regression. |

| Scenario                       | Response                                                                                                                                                     |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ledger reconciliation mismatch | <b>Sev-2.</b> Freeze the affected account's spending, replay the ledger to find the divergence, correct with a compensating entry, never by editing history. |
| Matchmaking p99 above SLA      | Check queue depth and concurrency. If concurrency is the cause, lower the bot-backfill threshold; if node placement is the cause, add match nodes.           |
| Suspected cheating report      | Pull the match's command log. Every accepted command is recorded, so the claim is checkable rather than arguable.                                            |
| Deploy interrupts matches      | Should not occur — nodes drain. If it does, the SERVER_DRAINING close code is missing and clients are backing off instead of reconnecting.                   |